ANSIBLE_SYSTEM_ROLES:

- **Ansible System Roles** are a special collection of **pre-built, supported, and standardized roles** designed to configure Linux systems consistently.
- **They're especially common on RHEL, CentOS Stream, Rocky, AlmaLinux, and other Enterprise Linux systems.**
- Beginning with Red Hat Enterprise Linux 7.4, a number of Ansible roles have been provided with the operating system as part of the rhel-system-roles package.
- In Red Hat Enterprise Linux 8 the package is available in the AppStream channel.
- As an example, the recommended time synchronization service for Red Hat Enterprise Linux 7 is the chronyd service.
- In Red Hat Enterprise Linux 6 however, the recommended service is the ntpd service.
- In an environment with a mixture of Red Hat Enterprise Linux 6 and 7 hosts, an administrator must manage the configuration files for both services.
- With RHEL System Roles, administrators no longer need to maintain configuration files for both services.
- Administrators can use rhel-system-roles.timesync role to configure time synchronization for both Red Hat Enterprise Linux 6 and 7 hosts.
- RHEL System Roles are derived from the open-source Linux System Roles project, found on Ansible Galaxy.

What Are Ansible System Roles?

Ansible System Roles are **ready-made roles** created by Red Hat to automate common system administration tasks. They follow best practices and work across multiple RHEL-based distributions.

They save you from writing roles from scratch for things like:

- Network configuration
- Storage setup
- Time synchronization
- SELinux
- Firewall
- Logging
- Kernel settings
- Postfix
- Kdump
- Certificate management

Where Do They Come From?

On RHEL-based systems, they are installed via:

```
sudo yum install rhel-system-roles
```

On Rocky/AlmaLinux, the package is usually:

```
sudo yum install linux-system-roles
```

This installs roles under:

/usr/share/ansible/roles/

- The default **roles_path** on Red Hat Enterprise Linux includes **/usr/share/ansible/roles** in the path, so Ansible should automatically find those roles when referenced by a playbook.
- Ansible might not find the system roles if roles_path has been overridden in the current Ansible configuration file, if the environment variable ANSIBLE_ROLES_PATH is set, or if there is another role of the same name in a directory listed earlier in roles_path.
- After installation, documentation for the RHEL System Roles is found in the /usr/share/doc/ rhel-system-roles-<version>/ directory. Documentation is organized into subdirectoriesby subsystem.

```
ls -l /usr/share/doc/rhel-system-roles/
```

- Each role's documentation directory contains a README.md file. The README.md file contains a description of the role, along with role usage information.
- The README.md file also describes role variables that affect the behavior of the role. Often the README.md file contains a playbook snippet that demonstrates variable settings for a common configuration scenario.

Common System Roles

Here are the most widely used ones:

Role Name	Purpose
network	Configure interfaces, bonds, VLANs, bridges
timesync	Configure NTP/chrony
selinux	Manage SELinux modes and policies
firewall	Configure firewalld rules
storage	Manage disks, partitions, LVM, filesystems
logging	Configure rsyslog
postfix	Configure mail relay
kernel_settings	Tune sysctl and kernel parameters
certificate	Manage TLS certificates
kdump	Configure kdump crash recovery

These roles are extremely powerful and save hours of manual YAML writing.

Example: Using the timesync System Role

1: Create group_vars/webservers/.yml

```yaml
timesync_ntp_servers:
  - hostname: 0.asia.pool.ntp.org
    iburst: yes
  - hostname: 1.asia.pool.ntp.org
    iburst: yes
  - hostname: 2.asia.pool.ntp.org
    iburst: yes
  - hostname: 3.asia.pool.ntp.org
    iburst: yes
timezone: Asia/Riyadh
```

2: PlayBook:

```yaml
- name: Time Synchronization Play
  hosts: webservers
  become: yes
  roles:
    - rhel-system-roles.timesync
  post_tasks:
    - name: Set timezone to Asia/Riyadh
      timezone:
        name: "{{ timezone }}"
      notify: restart chronyd
  handlers:
    - name: restart chronyd
      service:
        name: chronyd
        state: restarted
```

Example: Using the network System Role

```yaml
- name: Configure network interface
  hosts: all
  become: yes
  roles:
    - rhel-system-roles.network
  vars:
    network_connections:
      - name: ens33
        type: ethernet
        ip:
          address:
            - 192.168.1.10/24
          gateway: 192.168.1.1
          dns:
            - 8.8.8.8
```

CREATING ROLES

THE ROLE CREATION PROCESS Creating roles in Ansible requires no special development tools. Creating and using a role is a three-step process:

1. Create the role directory structure.
2. Define the role content.
3. Use the role in a playbook.

Creating a Role Skeleton

You can create all the subdirectories and files needed for a new role using standard Linux commands. Alternatively, command line utilities exist to automate the process of new role creation.

The ansible-galaxy command line tool is used to manage Ansible roles, including the creation of new roles.

You can run ansible-galaxy init to create the directory structure for a new role.

```
ansible-galaxy init xyz-role
```

CREATING THE ROLE DIRECTORY STRUCTURE

By default, Ansible looks for roles in a subdirectory called roles in the directory containing your Ansible Playbook. This allows you to store roles with the playbook and other supporting files.

If Ansible cannot find the role there, it looks at the directories specified by the Ansible configuration setting roles_path, in order. This variable contains a colon-separated list of directories to search.

The default value of this variable is: ~/.ansible/roles:/usr/share/ansible/roles:/etc/ansible/roles

This allows you to install roles on your system that are shared by multiple projects. For example, you could have your own roles installed your home directory in the ~/.ansible/roles subdirectory, and the system can have roles installed for all users in the /usr/share/ansible/ roles directory.

Each role has its own directory with a standardized directory structure.

DEFINING THE ROLE CONTENT

Once you have created the directory structure, you must write the content of the role. A good place to start is the ROLENAME/tasks/main.yml task file, the main list of tasks run by the role.

Roles allow playbooks to be written modularly. To maximize the effectiveness of newly developed roles, consider implementing the following recommended practices into your role development:

• Maintain each role in its own version control repository. Ansible works well with git-based repositories.

• Sensitive information, such as passwords or SSH keys, should not be stored in the role repository. Sensitive values should be parameterized as variables with default values that are not sensitive. Playbooks that use the role are responsible for defining sensitive variables through Ansible Vault variable files, environment variables, or other ansible-playbook options.

• Use ansible-galaxy init to start your role, and then remove any directories and files that you do not need.

• Create and maintain README.md and meta/main.yml files to document what your role is for, who wrote it, and how to use it.

• Keep your role focused on a specific purpose or function. Instead of making one role do many things, you might write more than one role.

• Reuse and refactor roles often. Resist creating new roles for edge configurations. If an existing role accomplishes a majority of the required configuration, refactor the existing role to integrate the new configuration scenario. Use integration and regression testing techniques to ensure that the role provides the required new functionality and also does not cause problems for existing playbooks.

DEFINING ROLE DEPENDENCIES

Role dependencies allow a role to include other roles as dependencies. For example, a role that defines a documentation server may depend upon another role that installs and configures a web server. Dependencies are defined in the meta/main.yml file in the role directory hierarchy.

Core Takeaway

Role dependencies let one role automatically pull in and execute other roles.
They are defined in meta/main.yml, run *before* the dependent role's tasks, and are executed only once unless allow_duplicates: yes is set.

What Role Dependencies Actually Do

A role can declare that it *requires* other roles to run first.

Example scenario:

- Your documentation_server role needs:
    - Apache installed
    - PostgreSQL configured

Instead of forcing the playbook author to remember to include those roles, you declare them as dependencies.

Where Dependencies Are Defined

Inside the role:

```
roles/
  documentation_server/
    meta/
      main.yml
```

Example meta/main.yml

Here is the structure you referenced, rewritten cleanly:

```
---
dependencies:
  - role: apache
    port: 8080
```

```
  - role: postgres
    dbname: serverlist
    admin_user: felix
```

What this means:

- Before documentation_server runs:
    - The apache role runs with port=8080
    - The postgres role runs with dbname=serverlist and admin_user=felix

---

Duplicate Dependency Behavior

By default:

- If multiple roles depend on the same role, **Ansible runs that dependency only once**.

This prevents:

- Duplicate package installs
- Duplicate service restarts
- Unnecessary overhead

Override this behavior:

```
allow_duplicates: yes
```

Use this **very carefully** — it can cause:

- Conflicting configurations
- Repeated service restarts
- Hard-to-debug behavior

Why You Should Limit Dependencies

Real-world reasons:

- Deep dependency chains become fragile
- Debugging becomes painful
- Versioning roles becomes harder
- Roles become less reusable
- You lose control over execution order

Best practice:
Use dependencies only when the role *cannot* function without the other role.

Use role dependencies only when:

- The dependent role *cannot* function without the other role
- The dependency is stable and version-controlled
- The dependency is small and predictable

Avoid dependencies when:

- The dependency is optional
- The dependency has many variables
- The dependency is large or complex
- You want the playbook author to control execution order

Using the Role in a Playbook

A simple playbook using the motd role:

```
---
- name: Configure message of the day
  hosts: all
  roles:
    - motd
```

Changing a Role's Behavior with Variables

A well-written role uses default variables to alter the role's behavior to match a related configuration scenario. This helps make the role more generic and reusable in a variety of contexts. The value of any variable defined in a role's defaults directory will be overwritten if that same variable is defined: • in an inventory file, either as a host variable or a group variable.

• in a YAML file under the group_vars or host_vars directories of a playbook project

• as a variable nested in the vars keyword of a play

• as a variable when including the role in roles keyword of a play

Variable precedence can be confusing when working with role variables in a play.

• Almost any other variable will override a role's default variables: inventory variables, play vars, inline role parameters, and so on.

• Fewer variables can override variables defined in a role's vars directory. Facts, variables loaded with include_vars, registered variables, and role parameters are some variables that can do that. Inventory variables and play vars cannot. This is important because it helps keep your play from accidentally changing the internal functioning of the role.

• However, variables declared inline as role parameters, have very high precedence. They can override variables defined in a role's vars directory. If a role parameter has the same name as a variable set in play vars, a role's vars, or an inventory or playbook variable, the role parameter overrides the other variable.

INTRODUCING ANSIBLE GALAXY Ansible

Galaxy [https://galaxy.ansible.com] is a public library of Ansible content written by a variety of Ansible administrators and users. It contains thousands of Ansible roles and it has a searchable database that helps Ansible users identify roles that might help them accomplish an administrative task. Ansible Galaxy includes links to documentation and videos for new Ansible users and role developers.

The ansible-galaxy command line tool :can be used to search for, display information about, install, list, remove, or initialize roles.

- The **ansible-galaxy search** subcommand searches Ansible Galaxy for roles.

  ansible-galaxy search 'redis' --platforms EL

- The **ansible-galaxy info** subcommand displays more detailed information about a role.

  ansible-galaxy info geerlingguy.redis

- The ansible-galaxy install subcommand downloads a role from Ansible Galaxy, then installs it locally on the control node.

- By default, roles are installed into the first directory that is writable in the user's **roles_path.**
  - Based on the default roles_path set for Ansible, normally the role will be installed into the user's ~/.ansible/roles directory.

- The default roles_path might be overridden by your current Ansible configuration file or by the environment variable ANSIBLE_ROLES_PATH, which affects the behavior of ansible-galaxy.

- You can also specify a specific directory to install the role into by using the -p DIRECTORY option.

  ansible-galaxy install geerlingguy.redis -p roles/

- You can also use ansible-galaxy to install a list of roles based on definitions in a text file.

- For example, if you have a playbook that needs to have specific roles installed, you can create a roles/requirements.yml file in the project directory that specifies which roles are needed.

  - src: geerlingguy.redis

    version: "1.5.0"

- *You should specify the version of the role in your requirements.yml file, especially for playbooks in production.*

- *If you do not specify a version, you will get the latest version of the role. If the upstream author makes changes to the role that are incompatible with your playbook, it may cause an automation failure or other problems*

- To install the roles using a role file, use the -r REQUIREMENTS-FILE option.

  ansible-galaxy install -r roles/requirements.yml

- The src keyword specifies the Ansible Galaxy role name. If the role is not hosted on Ansible Galaxy, the src keyword indicates the role's URL.

```
- src: geerlingguy.nginx
```

- If the role is hosted in a source control repository, the scm attribute is required.

```
- src: https://github.com/geerlingguy/ansible-role-nginx.git
  scm: git
```

- The ansible- galaxy command is capable of downloading and installing roles from either a Git-based or mercurial-based software repository.
- A Git-based repository requires an scm value of git, while a role hosted on a mercurial repository requires a value of hg.

```
- src: https://hg.example.com/ansible/roles/myrole
  scm: hg
```

- If the role is hosted on Ansible Galaxy or as a tar archive on a web server, the scm keyword is omitted.

```
- src: https://example.com/roles/myrole.tar.gz
```

- The **name** keyword is used to override the local name of the role.

```
- src: geerlingguy.nginx
  name: nginx
```

- The **version** keyword is used to specify a role's version. The version keyword can be any value that corresponds to a branch, tag, or commit hash from the role's software repository.

```
version: "1.2.0"
version: "stable"
version: "a8f3c9d"
```

- The ansible-galaxy list command can also manage local roles, such as those roles found in the roles directory of a playbook project.
- A role can be removed locally with the ansible-galaxy remove subcommand.

```
ansible-galaxy remove nginx-acme-ssh
```